

23-ma'ruza. Kortejlar. Sanab o'tiluvchi tur.

Reja:

1. Kortejlar haqida
2. Metod natijasi va parametri sifatida
3. Sanab o'tiluvchi tur: enum

Kortejlar C# dasturlash tilining 7.0 versiyasidan boshlab qo'llanila boshlagan va qiymatlar to'plami bilan ishlashning qulay usulini ta'minlaydi.

Qavslar ichiga olingan qiymatlar to'plamini ifodalaydi.

```
var kortej = (5, 10);
```

Bunday holda, ikkita qiymatga ega bo'lgan kortej aniqlanadi: 5 va 10. Bundan tashqari ushbu qiymatlarning har biriga nomli maydonlar orqali kirish mumkin:

```
Item[kortejdagi_maydonning_seriya_raqami]
```

Masalan:

```
var kortej = (5, 10);
```

```
Console.WriteLine(kortej.Item1); // 5
```

```
Console.WriteLine(kortej.Item2); // 10
```

```
kortej.Item1 += 26;
```

```
Console.WriteLine(kortej.Item1); // 31
```

Bunday holda, tur bevosita aniqlanadi. Lekin kortej o'zgaruvchisiga turni ham aniq berish mumkin:

```
(int, int) kortej = (5, 10);
```

Kortej ikkita sonni o'z ichiga olganligi sababli, tur ta'rifida ikkita sonli turni ko'rsatish kerak, kortejlarga yana bir misol:

```
(string, int, double) talaba = ("Anvar", 25, 81.23);
```

Bunda kortejning maydonlarini ham nomlash mumkin:

```
var kortej = (count: 5, sum: 10);
```

```
Console.WriteLine(kortej.count); // 5
```

```
Console.WriteLine(kortej.sum); // 10
```

Endi kortej maydonlariga murojaat qilish uchun Item1 va Item2 nomlari emas, balki ularning nomlari ishlatiladi.

Hatto kortejni alohida o'zgaruvchilarga ajratish mumkin:

```
var (name, age) = ("Nodir", 23);
```

```
Console.WriteLine(name); // Nodir
```

```
Console.WriteLine(age); // 23
```

Kortej hal qila oladigan vazifalardan biri bu qiymatlar almashinuvidir:

```
string main = "Java";
```

```
string second = "C#";
```

```
(main, second) = (second, main);
```

```
Console.WriteLine(main); // C#
```

```
Console.WriteLine(second); // Java
```

Masalan, massivni eng oddiy tartiblash uchun kortejdan foydalanish mumkin:

```
using System;
```

```
namespace ConsoleApp5
```

```
{
```

```
    internal class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            int[] nums = { 54, 7, -41, 2, 4, 2, 89, 33, -5, 12 };
```

```
            // tartiblash
```

```
            for (int i = 0; i < nums.Length - 1; i++)
```

```

    {
        for (int j = i + 1; j < nums.Length; j++)
        {
            if (nums[i] > nums[j])
                (nums[i], nums[j]) = (nums[j], nums[i]);
        }
    }
    // chiqarish
    Console.WriteLine("Saralangan massiv chiqishi:");
    for (int i = 0; i < nums.Length; i++)
    {
        Console.Write(nums[i] + " ");
    }
    Console.ReadKey();
}
}

```

Dastur natijasi:

Saralangan massiv chiqishi:

-41 -5 2 2 4 7 12 33 54 89

Kortejlar metod natijasi sifatida

Kortejlar metod natijasi sifatida ham ishlatilishi mumkin. Masalan, metod ikki yoki undan ortiq qiymatlarni qaytarsa, metod faqat bitta qiymatni qaytarishi mumkin va kortejlar ushbu vazifani bajarishning eng maqbul usulini ifodalaydi:

```

using System;
namespace ConsoleApp5
{
    internal class Program
    {
        static void Main(string[] args)
        {
            var kortej = GetValues();
            Console.WriteLine(kortej.Item1); // 1
            Console.WriteLine(kortej.Item2); // 3
            Console.ReadKey();
        }
        static (int, int) GetValues()
        {
            var result = (1, 3);
            return result;
        }
    }
}

```

Dastur natijasi:

1

3

Bu yerda, GetValues() metodi aniqlanadi, bu kortejni qaytaradi. Kortejlar qavslar ichiga olingan qiymatlar to'plami sifatida aniqlanadi va bu holda int turidagi ikkita elementdan iborat kortejni, ya'ni ikkita raqamni qaytaradi.

Yana bir misol:

```

using System;
namespace ConsoleApp5

```

```

{
    internal class Program
    {
        static void Main(string[] args)
        {
            var kortej = GetValuesData(new int[] { 1, 2, 3, 4, 5, 6, 7 });
            Console.WriteLine(kortej.count);
            Console.WriteLine(kortej.sum);
            Console.ReadKey();
            Console.ReadKey();
        }
        static (int sum, int count) GetValuesData(int[] numbers)
        {
            var result = (sum: 0, count: numbers.Length);
            foreach (var n in numbers)
            {
                result.sum += n;
            }
            return result;
        }
    }
}

```

Dastur natijasi:

7

28

Metod parametri sifatida kortej

Shuningdek, kortejni metodga parametr sifatida o'tkazish mumkin:

```
PrintPerson(("Nodir", 33)); // Nodir - 33
```

```
PrintPerson(("Oybek", 30)); // Oybek - 30
```

```
Console.ReadKey();
```

```
void PrintPerson((string name, int age) person)
```

```

{
    Console.WriteLine($"{person.name} - {person.age}");
}

```

Bu yerda, PrintPerson metodiga ikkita elementdan iborat kortej uzatiladi, ularning birinchisi satrni, ikkinchisi esa int qiymatini beradi.

Sanab o'tiluvchi tur: enum

C# dasturlash tilida oddiy ma'lumotlar turlaridan tashqari, enum ya'ni ro'yxat kabi turlari mavjud. Sanab o'tiluvchi turlar mantiqiy konstanta to'plamini ifodalaydi.

Sanab o'tiluvchi turlar **enum** kalit so'zi bilan e'lon qilinadi.

```
enum royxat_nomi
```

```

{
    qiymat1,
    qiymat2,
    .....
    qiymatN
}

```

enum e'lonida enum kalit so'zidan keyin sanab o'tiluvchi ro'yxat nomi keladi. Keyin ro'yxat konstantalari vergul bilan ajratilgan holda {} qavslar ichida keltiriladi.

Keling, eng oddiy ro'yxatni aniqlaymiz:

```
enum DayTime
```

```

{
    Morning,
    Afternoon,
    Evening,
    Night
}

```

DayTime ro'yxati e'lon qilinadi va to'rtta qiymatga ega: Morning, Afternoon, Evening va Night.

Har bir ro'yxatga olish haqiqatan ham yangi ma'lumotlar turini belgilaydi, uning yordamida, xuddi har qanday boshqa turdagi kabi, o'zgaruvchilarni, konstantalarni, metod parametrlarini va hokazolarni aniqlash mumkin. Sanoqni ifodalovchi o'zgaruvchi, doimiy va metod parametrining qiymati sanab o'tilgan konstantalardan biri bo'lishi kerak, masalan:

```
const DayTime dayTime = DayTime.Morning;
```

Keyinchalik dasturda, boshqa har qanday o'zgaruvchilar kabi o'zgaruvchilar/konstantalar/parametrlardan foydalanish mumkin:

```
DayTime dayTime = DayTime.Morning;
```

```
if (dayTime == DayTime.Morning)
```

```
    Console.WriteLine("Xayrli tong ");
```

```
else
```

```
    Console.WriteLine("Salom");
```

```
enum DayTime
```

```
{
```

```
    Morning,
```

```
    Afternoon,
```

```
    Evening,
```

```
    Night
```

```
}
```

Holatni saqlash

Ko'pincha, *enum* o'zgaruvchisi ba'zi harakatlar bajarilishiga qarab, holatni saqlash vazifasini ham bajaradi:

```
DayTime now = DayTime.Evening;
```

```
PrintMessage(now);
```

```
PrintMessage(DayTime.Afternoon);
```

```
PrintMessage(DayTime.Night);
```

```
void PrintMessage(DayTime dayTime)
```

```
{
```

```
    switch (dayTime)
```

```
    {
```

```
        case DayTime.Morning:
```

```
            Console.WriteLine("Xayrli tong");
```

```
            break;
```

```
        case DayTime.Afternoon:
```

```
            Console.WriteLine("Xayrli kun");
```

```
            break;
```

```
        case DayTime.Evening:
```

```
            Console.WriteLine("Xayrli kech");
```

```
            break;
```

```
        case DayTime.Night:
```

```
            Console.WriteLine("Xayrli tun");
```

```
            break;
```

```
    }
```

```
}
```

```
enum DayTime
{
    Morning,
    Afternoon,
    Evening,
    Night
}
```

Bu yerda, PrintMessage() metodi parametr sifatida DayTime sanab o'tiluvchi turining qiymatini oladi va shu qiymatga qarab ma'lum bir jarayonni chiqaradi.

Yana bitta misol:

```
DoOperation(10, 5, Operation.Add);           // 15
DoOperation(10, 5, Operation.Subtract);      // 5
DoOperation(10, 5, Operation.Multurly);      // 50
DoOperation(10, 5, Operation.Divide);        // 2
void DoOperation(double x, double y, Operation op)
{
    double result = op switch
    {
        Operation.Add => x + y,
        Operation.Subtract => x - y,
        Operation.Multurly => x * y,
        Operation.Divide => x / y
    };
    Console.WriteLine(result);
}
enum Operation
{
    Add,
    Subtract,
    Multurly,
    Divide
}
```

Bu arifmetik amallarni ifodalovchi ro'yxatni belgilaydi. Har bir operatsiya turi sanab o'tish konstantalaridan biri sifatida aniqlanadi. DoOperation() metodi aniqlanadi, u parametrlar sifatida enum sifatida ikkita son va operatsiya turini oladi va shu turga qarab switch konstruktsiyasidan ma'lum bir operatsiya natijasini qaytaradi.

enum konstantalarining turi va qiymatlari

enum konstantalari turga ega bo'lishi mumkin. Tur sanab o'tilgan nomdan keyin ko'rsatiladi, ikki nuqta bilan ajratiladi:

```
enum Time : byte
{
    Morning,
    Afternoon,
    Evening,
    Night
}
```

Sanoq turi butun son turini ifodalashi kerak (byte, sbyte, short, ushort, int, uint, long, ulong). Agar tur aniq ko'rsatilmagan bo'lsa, standart tur int bo'ladi.

Ro'yxatdagi har bir elementga butun son qiymati beriladi, birinchi element 0, ikkinchisi 1 va hokazo. Masalan, yuqorida belgilangan DayTime ni ko'ramiz:

```
DayTime now = DayTime.Morning;
Console.WriteLine((int)now); // 0
```

```

Console.WriteLine((int)DayTime.Night); // 3
enum DayTime
{
    Morning,
    Afternoon,
    Evening,
    Night
}
enum konstantasining butun qiymatini olish uchun quyidagi operatoridan foydalanish
mumkin:
(int) DayTime.Night // 3
Shu bilan birga, har bir konstanta ma'lum bir songa mos kelsa ham, unga sonli qiymat berib
bo'lmaydi:
DayTime now = 2; // ! Xato
Bundan tashqari, element birinchi elementning qiymatini belgilash mumkin:
enum DayTime
{
    Morning = 3, // har bir keyingi element standart bittaga oshiriladi
    Afternoon, // bu element 4 ga teng
    Evening, // 5
    Night // 6
}
yoki barcha elementlar uchun qiymatlarni ham belgilash mumkin:
enum DayTime
{
    Morning = 2,
    Afternoon = 4,
    Evening = 8,
    Night = 16
}
Bunday holda, sanab o'tiluvchi konstantalar bir xil qiymatlarga ega bo'lishi mumkin yoki
bitta o'zgarishga boshqa o'zgarishning qiymati ham berilishi mumkin:
enum DayTime
{
    Morning = 1,
    Afternoon = Morning,
    Evening = 2,
    Night = 2
}

```